COSMOS][- 言語仕様

アーカイブ	☐
編集日	@2026年3月3日 7:46

独自言語仕様 — 対話型インタプリタ実行環境（COSMOS II）【32bit符号付整数版】

以下の独自言語仕様に基づいて「対話型インタプリタ実行環境（COSMOS II）」を構築する。

> [!IMPORTANT]
> 本仕様は**32bit符号付整数版**である。乗除算が加減算より優先される。数値型は **32bit 符号付整数（-2147483648 〜 2147483647）** である。比較演算子の等価は `==` を使用する（代入 `=` との区別を明確化）。

---

1. データモデル

1.1 数値型

- すべての値は **32bit 符号付整数（-2147483648 〜 2147483647）** として扱う。
- 内部表現は**2の補数**形式とする。
- 演算結果が範囲外となる場合は **32bit幅でラップアラウンド**（2の補数のオーバーフロー動作）する。
  - 例: `2147483647 + 1 = -2147483648` / `2147483648 - 1 = 2147483647`

1.2 変数

種別	識別子	属性	初期値
ユーザー変数	`A` 〜 `Z`, `a` 〜 `z`（計52個、1文字のみ識別）	読み書き	`0`（システム起動時のみ初期化）
システム変数 `%`	直近の除算 `/` における剰余	読み取り専用	`0`（システム起動時のみ初期化）
システム変数 `'`	参照のたびに -2147483648〜2147483647 の乱数を生成	読み取り専用	不定（動的生成）

> [!IMPORTANT]
> システム変数 `%`, `'` への代入はエラーとする。
> ユーザー変数およびシステム変数 `%` はシステム起動時にのみ `初期値` で初期化される。プログラム実行時（RUN や GOTO / GOSUB による実行開始を含む）には初期化されない。明示的に初期化するには `<変数初期化文>` 文を使用する。

1.3 配列（メモリ）空間とアドレス

インタープリタは配列参照のために論理的なメモリアドレス空間（0〜65535）を提供する。
・変数へのバイト/ワード/ダブルワード参照は、変数の値（下位16bit）を無符号アドレスとして解釈し、論理メモリ空間にアクセスする。
・内部のリトル/ビッグエンディアンや変数の物理的な配置アドレスは実行環境に依存する（参照：実装仕様）。

---

2. 字句定義（Lexical）

2.1 トークン種別

トークン	表記	規則
10進定数	`0` 〜 `2147483647`	数字のみで構成。負の定数はリテラルとしては存在せず、単項 `-` で表現する（例: `-1`）。
16進定数	`$` + 16進数字	1〜8桁、`0`-`9`, `A`-`F`（大文字・小文字混在可）。値は2の補数として解釈する（例: `$FFFFFFFF` = -1）。
2進定数	`%` + 2進数字	1〜32桁、`0` または `1` のみ。値は2の補数として解釈する。
文字定数	`"` + ASCII文字 + `"`	評価値はその文字のASCIIコード（0〜127）。
変数名	`A` 〜 `Z`, `a` 〜 `z`	1文字のみ識別（2文字目以降は無視）
システム変数	`%`, `'`	§1.2 参照
演算子	`+`, `-`, `*`, `/`, `==`, `<>`, `<`, `>`, `<=`, `>=`, `&&`, `||`, `^`, `<<`, `>>`, `#`, `##`	§4 参照
区切り記号	`(`, `)`, `[`, `]`, `{`, `}`, `:`, `,`, `;`, `@`, `!`, `?`, `$`, `.`, `/`, `"`	文脈により意味が変わる

2.2 スペースの扱い

- ダブルクォート `"..."` の内側に現れるスペースは、**リテラルの空白文字**としてそのまま扱う。
- ダブルクォートの外側に現れるスペース（1文字以上の連続）は、**文と文の区切り**（ステートメントセパレータ）として扱う。
- 1つの文（statement）の中にスペースを含めてはならない。

> [!NOTE]
> トークナイザはダブルクォート外の連続するスペースを1つのステートメントセパレータとして扱い、式評価器はスペースをトークンとして認識しないため、スペースをまたいだ式の記述は文法エラーとなる。

2.3 `%` の曖昧性解決

`%` は2進定数のプレフィクスとシステム変数の2つの意味を持つ。以下の規則で判別する：

- `%` の直後が `0` または `1` → **2進定数**として解釈
- `%` の直後が `0`, `1` 以外（空白、演算子、行末等）→ **システム変数**として解釈

---

3. プログラム構造（Program Structure）

3.1 行の構造

```
<行>        ::= <行番号> " " <行本体> <改行>
<行番号>    ::= "1" 〜 "32767"
<改行>      ::= "\\n" | "\\r\\n"
<行本体>    ::= <文の並び> | <コメント>
<文の並び> ::= <文> { <空白> <文> }
<空白>      ::= { " " }+
```

3.2 コメントの判定

行本体の中に以下のいずれかの文字列が現れた場合、その文字列以降、行末までをコメントとして扱い、評価・実行をスキップする（インラインコメント対応）。

- `//`
- `##`
- `-`

---

4. 構文定義と動作仕様

4.1 式と演算子

構文（演算子優先順位を反映したBNF）

```
<式>          ::= <ビットOR式> { "||" <ビットOR式> }

<ビットOR式>  ::= <ビットXOR式> { "^" <ビットXOR式> }

<ビットXOR式> ::= <ビットAND式> { "&&" <ビットAND式> }

<ビットAND式> ::= <比較式> { <比較演算子> <比較式> }

<比較式>      ::= <シフト式> { <シフト演算子> <シフト式> }

<シフト式>    ::= <加減式> { ("+" | "-") <加減式> }

<加減式>      ::= <乗除式> { ("*" | "/") <乗除式> }

<乗除式>      ::= <単項式>

<単項式>      ::= <単項演算子> <単項式>
                | <一次式>

<一次式>      ::= <定数>
                | <変数名>
                | <配列参照>
                | <システム変数>
                | "(" <式> ")"
```

```
                  | "?"              (* 10進数値入力 *)
                  | "$"              (* 1文字入力 *)

<定数>          ::= <10進定数> | <16進定数> | <2進定数> | <文字定数>
```

演算子の優先順位（高い順）

優先度	演算子	結合規則	説明
1（最高）	`()`	—	括弧（グループ化）
2	`+` `-` `#` `##`	右→左	単項演算子
3	`*` `/`	左→右	乗算・除算
4	`+` `-`	左→右	加算・減算
5	`<<` `>>`	左→右	シフト演算
6	`==` `<>` `<` `>` `<=` `>=`	左→右	比較演算
7	`&&`	左→右	ビットワイズ AND
8	`^`	左→右	ビットワイズ XOR
9（最低）	`||`	左→右	ビットワイズ OR

例:

- `1 + 2 * 3` → `1 + (2 * 3)` → `7`
- `2 * 3 + 4 * 5` → `(2 * 3) + (4 * 5)` → `26`
- `10 - 4 / 2` → `10 - (4 / 2)` → `8`
- `A == 1 && B == 1` → `(A == 1) && (B == 1)`（両条件のビットAND）
- `3 * 2` → `(-3) * 2` → `6`（単項演算子は乗除より優先）

二項演算子

```
<二項演算子> ::= <算術演算子> | <比較演算子> | <ビット演算子> | <シフト演算子>
```

算術演算子

構文	優先度	動作
`*`	3	乗算。結果は32bit符号付でラップアラウンド。
`/`	3	符号付整数除算（ゼロへの丸め）。実行直後、システム変数 `%` に剰余を格納する。剰余の符号は被除数と同じ。右辺が `0` の場合は実行時エラー（実装仕様参照）。例: `7 / 2 = 3`（`% = 1`）、`-7 / 2 = -3`（`% = -1`）。
`+`	4	加算。結果は32bit符号付でラップアラウンド。
`-`	4	減算。結果は32bit符号付でラップアラウンド。

比較演算子

構文	優先度	動作
`==`	6	等しければ `1`、そうでなければ `0`
`<>`	6	等しくなければ `1`、そうでなければ `0`

構文	優先度	動作
`<`	6	左辺 < 右辺 なら `1`、そうでなければ `0`（符号付比較）
`>`	6	左辺 > 右辺 なら `1`、そうでなければ `0`（符号付比較）
`<=`	6	左辺 <= 右辺 なら `1`、そうでなければ `0`（符号付比較）
`>=`	6	左辺 >= 右辺 なら `1`、そうでなければ `0`（符号付比較）

ビット演算子

構文	優先度	動作
`&&`	7	ビットワイズ AND
`^`	8	ビットワイズ XOR
`||`	9	ビットワイズ OR

シフト演算子

構文	優先度	動作
`<<`	5	左シフト。結果は32bit符号付でラップアラウンド。
`>>`	5	算術右シフト。符号ビットを維持しながら右シフトする。例: `-4 >> 1 = -2`。

単項演算子（優先度2）

構文	動作
`+`	絶対値。負の値は正に変換する。例: `+(-5) = 5`。ただし `+(-2147483648) = -2147483648`（表現不可のためラップアラウンド）。
`-`	符号反転。`0 - x` として計算（2の補数）。例: `-(5) = -5`。ただし `-(-2147483648) = -2147483648`（ラップアラウンド）。
`#`	論理否定。値が `0` なら `1`、それ以外なら `0`。
`##`	ビット反転。全ビットを反転する。例: `##0 = -1`、`##(-1) = 0`。

4.2 配列参照

```
<配列参照>          ::= <バイト参照> | <ワード参照> | <ダブルワード参照>
<バイト参照>        ::= <変数名> "[" <式> "]"
<ワード参照>        ::= <変数名> ":" <式> "]"
<ダブルワード参照> ::= <変数名> "{" <式> "}"
```

種別	アドレス計算	アクセス単位
バイト参照 `V[n]`	`V` の値 + `n`	1バイト（読み出し: 0〜255、書き込み: 下位8bit）
ワード参照 `V:n]`	`V` の値 + `n` × 2	2バイト（符号付16bit、ビッグエンディアン）
ダブルワード参照 `V{n}`	`V` の値 + `n` × 4	4バイト（符号付32bit、ビッグエンディアン）

アドレス計算は無符号として行う（変数値のビットパターンを無符号アドレスとして使用）。算出アドレスが `0`〜`65535` の範囲外の場合は実行時エラー（実装仕様参照）。ワード参照の場合は算出アドレスから2バイト分、ダブルワード参照の場合は算出アドレスから4バイト分が範囲内に収まる必要がある。

4.3 代入文

```
<代入文>    ::= <代入対象> "=" <式>
<代入対象> ::= <変数名> | <配列参照>
```

4.4 制御文

```
<制御文> ::= "#" "=" <式>    (* GOTO *)
           | "!" "=" <式>   (* GOSUB *)
           | "]"            (* RETURN *)
           | ";" "=" <式>   (* IF *)
```

文	動作
`# = <式>` (GOTO)	式の値が `-1`（内部表現 `$FFFFFFFF`）ならプログラム終了（END）。それ以外は、式の値を行番号として解釈し、指定行番号へジャンプする。指定行番号が存在しない場合、それより大きい最小の行番号へジャンプする。該当する行が存在しない場合は実行時エラー（実装仕様参照）。
`! = <式>` (GOSUB)	現在の実行位置（行番号＋文インデックス）をCallStackに保存し、GOTO と同じ規則で指定行へジャンプする。
`]` (RETURN)	CallStack から戻り先を取り出し、呼び出し元の次の文から実行を再開する。スタックが空の場合は実行時エラー（実装仕様参照）。
`; = <式>` (IF)	式の結果が `0`（偽）の場合、現在の行の残りの文をすべてスキップし、次の行へ移行する。`0` 以外（真）の場合は何もせず次の文を実行する。

> [!NOTE]
> 行番号は `1`〜`32767` であるため、GOTO/GOSUB の式の値が負数（-1 以外）や 0 の場合、有効な行番号には該当しない。`-1` は特別にプログラム終了として扱う。

4.5 ループ文

```
<ループ文> ::= "@"                          (* DO: ループ開始点 *)
             | "@" "=" "(" <式> ")"         (* UNTIL: 条件真で終了 *)
             | <代入対象> "=" <式> "," <式> (* FOR: 初期値, ステップ *)
             | "@" "=" <式>                 (* NEXT: 上限値 *)
```

DO 〜 UNTIL

1. `@` — 現在の実行位置（行番号＋文インデックス）を LoopStack に保存する。
2. `@=(<式>)` — 式を評価し、結果が `0`（偽）なら LoopStack の先頭が指す `@` の次の文に戻る。`0` 以外（真）なら LoopStack から先頭を取り除き、次の文へ進む（ループ終了）。

FOR 〜 NEXT

1. `<変数> = <初期値>, <ステップ>` — 変数に初期値を代入し、ステップ値を内部に記憶する。同時に、現在の実行位置を LoopStack に保存する。

2. `@ = <上限値>` — 変数にステップ値を加算する。加算後の変数値が上限値を超えていなければ、FOR 設定行の次の文に戻る。超えていればループ終了（LoopStack から先頭を取り除き、次の文へ進む）。

> [!NOTE]
> 「超えている」の判定はステップ値の符号で決まる。ステップが正なら `変数 > 上限値` で終了、ステップが負なら `変数 < 上限値` で終了。

> [!NOTE]
> パーサは代入文のパース時に `,` が後続すれば FOR 文として判別する。

4.6 変数初期化文

```
<変数初期化文> ::= "@@"
```

文	動作
`@@`	ユーザー変数（`A`〜`Z`, `a`〜`z`）およびシステム変数 `%` をすべて `初期値` にリセットする。

4.7 入出力文

```
<入出力文> ::= "?" "=" <式>                    (* 10進出力 *)
             | "??" "=" <式>                   (* 16進4桁出力 *)
             | "?$" "(" <数字> ")" "=" <式>    (* 16進n桁出力 *)
             | "?$" "=" <式>                   (* 16進2桁出力 *)
             | "?0" "(" <数字> ")" "=" <式>    (* n桁右詰め出力、ゼロパディング *)
             | "?" "(" <数字> ")" "=" <式>     (* n桁右詰め出力、空白パディング *)
             | "$" "=" <式>                    (* 1文字出力 *)
             | "." "=" <式>                    (* 空白出力 *)
```

文	動作
`?=<式>`	式の値を符号付10進数で出力する。負の値は `-` 付きで表示される。例: `?=-5` → `-5`
`??=<式>`	式の値の下位16bitを16進4桁（ゼロパディング）で出力する。例: `??=-1` → `FFFF`
`?$(n)=<式>`	式の値の内部表現（符号なし32bit）を16進n桁（ゼロパディング）で出力する。n桁に収まらない場合は下位n桁を表示。例: `?$(8)=-1` → `FFFFFFFF`、`?$(4)=-1` → `FFFF`
`?$=<式>`	式の値の下位8bitを16進2桁（ゼロパディング）で出力する。例: `?$=-1` → `FF`
`?0(n)=<式>`	式の値を符号付10進n桁で右詰め出力する（ゼロパディング）。符号は桁数に含む。
`?(n)=<式>`	式の値を符号付10進n桁で右詰め出力する（空白パディング）。符号は桁数に含む。
`$=<式>`	式の値の下位8bitをASCII文字として1文字出力する。
`.=<式>`	式の値が正の場合、その数だけ半角スペースを出力する。`0` 以下の場合は何も出力しない。

4.8 文字列出力・改行出力

```
<文字列出力> ::= "\\"" <文字列> "\\""      (* ダブルクォート内の文字列をそのまま出力 *)
<改行出力>   ::= "/"                       (* 改行を出力 *)
```

4.9 文の定義

```
<文> ::= <代入文>
       | <制御文>
       | <ループ文>
       | <変数初期化文>
       | <入出力文>
       | <文字列出力>
       | <改行出力>
```